

Métricas técnicas de avaliação

Explicação do teste:

1. Ambiente Experimental e Configurações de Hardware

Os experimentos de bancada foram executados em ambiente computacional local sob as seguintes especificações de hardware e software:

Componente	Especificação Técnica	Papel no Experimento
Processador (CPU)	Intel Core i5-11400F (11ª Geração, 6 núcleos / 12 threads @ 2.60 GHz até 4.40 GHz)	Orquestração do pipeline, processamento dos embeddings vetoriais e inferência de apoio.
Placa de Vídeo (GPU)	NVIDIA GeForce RTX 4060 Ti (8 GB GDDR6 / 8.188 MiB VRAM dedicada)	Aceleração de operações paralelas e monitoramento de memória de vídeo dedicada.
Memória RAM	16 GB DDR4	Armazenamento do índice em memória, do runtime Python e

		do cache de inferência.
Sistema Operacional	Windows 11 (x64)	Plataforma base de execução.
Ambiente de Software	Python 3.11, PyTorch (v2.13.0+cpu), LangChain (v1.3.14), ChromaDB, Ollama Runtime	Ecossistema para orquestração de RAG e execução local dos modelos.

2. Descrição do Experimento Utilizado

Objetivo

Avaliar comparativamente o desempenho computacional (latência de inferência de ponta a ponta, pegada de memória RAM e VRAM) e a capacidade analítica de geração textual de três arquiteturas de Modelos de Linguagem de Grande Porte (LLMs) de código aberto:

- Meta LLaMA 3 (8B): Modelo de referência da proposta.
- Mistral (7B): Modelo comparativo de alta concisão.
- Google Gemma (7B): Modelo comparativo de arquitetura densa.

Arquitetura do Pipeline RAG Testado

Para que todos os modelos fossem avaliados em condições idênticas e controladas, o experimento submeteu cada modelo ao seguinte fluxo sequencial:

1. Entrada: Recepção da pergunta de teste.
2. Vetorização: Modelo BERTimbau-base (768d) gera o vetor da consulta.
3. Recuperação: Banco ChromaDB (Indexação MMR) recupera os top-4 *chunks* mais relevantes (fetch_k=25).
4. Construção de Prompt: Prompt Builder estruturado injeta Contexto + Histórico + Regras Anti-Alucinação.
5. Inferência: LLM no Ollama (Quantização 4-bit) processa a requisição.
6. Saída: Geração da resposta analítica e coleta de métricas (Tempo, RAM, VRAM, Palavras).

Parâmetros de Execução

- Corpus Base: Dados reais de postagens do TikTok brasileiro (postagens_tiktok.csv) contendo transcrições de áudio, resumos do conteúdo visual, hashtags e métricas de visualizações.
- Segmentação (Chunking): Segmentação textual via RecursiveCharacterTextSplitter com tamanho de bloco de 500 caracteres e 50 caracteres de sobreposição (*overlap*).
- Recuperação com Diversificação (MMR): Aplicação do algoritmo *Maximal Marginal Relevance* (k=4, fetch_k=25), garantindo que os vídeos recuperados não sejam cópias redundantes do mesmo criador.
- Isolamento de Hardware: Antes da troca de cada modelo, o script executa uma rotina preventiva de descarregamento da VRAM (função `limpar_memoria_gpu` com `keep_alive=0` e coleta de lixo `gc.collect()`), garantindo que um modelo não

aproveite tensores residuais do modelo anterior na memória.

Resultados do teste:

RESUMO CONSOLIDADO DE PERFORMANCE TÉCNICA E MEMÓRIA					
Modelo	Tempo Médio	VRAM Pico	RAM Média	Média Palavras	Sucesso
llama3	14.87s	1.36 GB	12.35 GB	42.7	3/3
mistral	13.02s	1.36 GB	12.10 GB	137.0	3/3
gemma:7b	12.99s	1.36 GB	13.33 GB	33.0	3/3

1. Análise Qualitativa por Modelo

- **Mistral (7B) — O mais analítico e completo**
 - Destaque: Entregou a maior utilidade para análise de tendências (média de 137 palavras, mais que o triplo do LLaMA 3).
 - Comportamento: Não se limitou a listar tópicos; explicou o contexto das músicas sertanejas românticas, detalhou os vídeos com métricas e justificou a razão do uso das hashtags (ex: contextualizando #TikTok e #futebol).
 - Velocidade: Tempo médio de 13,02s, mesmo gerando respostas substancialmente maiores.
- **Meta LLaMA 3 (8B) — O modelo oficial da proposta**
 - Destaque: Cumpre perfeitamente a regra de fidelidade estrita (sem alucinações).
 - Comportamento: Respostas curtas, objetivas e organizadas em listas de marcadores (*bullet points*).

Respondeu pontualmente à pergunta da métrica (identificando o vídeo de mais de 4 milhões de visualizações) e listou as hashtags reais sem rodeios.

- Velocidade: Tempo médio de 14,87s, mantendo 100% de estabilidade.
- **Google Gemma (7B) — Conservador e cauteloso**
 - Destaque: Concluiu as 3 baterias de perguntas sem travar (100% de sucesso).
 - Comportamento: Apresentou uma forte tendência a "falsos negativos" nas perguntas 2 e 3. Embora os dados estivessem no contexto recuperado (comprovado pelos acertos do LLaMA e Mistral), recuou afirmando: "A resposta fornecida não contém informações... portanto não posso responder".
 - Consumo: Foi o modelo que mais exigiu da memória RAM do sistema (pico de 13,33 GB).

2. Diagnóstico de Hardware (RAM e VRAM)

- **Uso de RAM do Sistema (12,1 GB a 13,3 GB):** Como o PyTorch do ambiente experimental está configurado para CPU, a inferência foi processada na memória RAM principal. O sistema operou com segurança, consumindo até ~13,3 GB dos 16 GB totais disponíveis, sem causar gargalos ou travamentos na máquina.
- **Uso de VRAM (1,36 GB):** O consumo da placa de vídeo manteve-se no nível basal do sistema operacional. Isso comprova que todo o processamento de geração foi orquestrado com sucesso na RAM, sem risco de estourar a memória dedicada da RTX 4060 Ti.

3. Conclusão do Experimento

- **Validação do Pipeline:** O fluxo da arquitetura RAG (integrando BERTimbau, ChromaDB e LLMs locais via Ollama) funcionou de ponta a ponta com 100% de taxa de sucesso de execução.
- **Alinhamento com a Proposta:** O LLaMA 3 (8B) demonstrou forte consistência e factualidade estrita, atendendo diretamente aos requisitos metodológicos estabelecidos no escopo do projeto.
- **Achado Científico Relevante:** O Mistral (7B) revelou-se um candidato excepcional para tarefas de sumarização e análise profunda de tendências em redes sociais em português, gerando relatórios textuais muito mais informativos e contextualizados para o usuário final do que o modelo base.